

# Insight-to-Action Engine

## Technical Implementation Guide

---

### Quick Start Development

#### Prerequisites

```
bash

# Python 3.12+
python --version

# Node.js 18+
node --version

# Docker
docker --version
docker-compose --version
```

#### Initial Setup

```
bash
```

```
# Clone and navigate
git clone <repo-url>
cd insight-to-action

# Install Python dependencies
cd apps/backend
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install -r requirements.txt

# Install Node dependencies
cd ../frontend-web
npm install

# Set up environment
cd ../../
cp .env.example .env
# Edit .env with your local credentials
```

## Start Local Environment

```
bash
```

```
# From project root
docker-compose up -d

# Run migrations
cd apps/backend
alembic upgrade head

# Start backend server
uvicorn app.main:app --reload

# In another terminal, start frontend
cd apps/frontend-web
npm run dev

# Access at http://localhost:3000
```

---

## Backend Setup Details

### FastAPI Main Application Structure

**apps/backend/app/main.py**

```
python
```

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.middleware.auth import JWTAuthMiddleware
from app.middleware.logging import LoggingMiddleware
from app.middleware.rate_limit import RateLimitMiddleware
from app.routers import auth, documents, insights, actions, websocket

app = FastAPI(
    title="Insight-to-Action Engine",
    version="1.0.0",
    docs_url="/api/docs"
)

# Middleware stack (order matters)
app.add_middleware(RateLimitMiddleware)
app.add_middleware(LoggingMiddleware)
app.add_middleware(JWTAuthMiddleware)
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:3000"])

# Include routers
app.include_router(auth.router, prefix="/api/auth", tags=["auth"])
app.include_router(documents.router, prefix="/api/documents", tags=["documents"])
app.include_router(insights.router, prefix="/api/insights", tags=["insights"])
app.include_router(actions.router, prefix="/api/actions", tags=["actions"])
app.include_router(websocket.router, prefix="/api/ws", tags=["websocket"])

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

```

## Authentication Service

**apps/backend/app/services/auth\_service.py**

python

```
import jwt
from datetime import datetime, timedelta
from passlib.context import CryptContext
from app.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

class AuthService:
    @staticmethod
    def hash_password(password: str) -> str:
        return pwd_context.hash(password)

    @staticmethod
    def verify_password(plain: str, hashed: str) -> bool:
        return pwd_context.verify(plain, hashed)

    @staticmethod
    def create_access_token(user_id: str) -> str:
        payload = {
            "sub": user_id,
            "exp": datetime.utcnow() + timedelta(minutes=15),
            "iat": datetime.utcnow()
        }
        return jwt.encode(payload, settings.SECRET_KEY, algorithm="HS256")

    @staticmethod
    def create_refresh_token(user_id: str) -> str:
        payload = {
            "sub": user_id,
            "exp": datetime.utcnow() + timedelta(days=7),
            "type": "refresh",
            "iat": datetime.utcnow()
        }
```

```
        return jwt.encode(payload, settings.SECRET_KEY, algorithm="HS256")

    @staticmethod
    def verify_token(token: str) -> dict:
        try:
            return jwt.decode(token, settings.SECRET_KEY, algorithms=["HS256"])
        except jwt.ExpiredSignatureError:
            return None
        except jwt.InvalidTokenError:
            return None
```

## Database Models

**apps/backend/app/models/user.py**

python

```
from sqlalchemy import Column, String, Boolean, DateTime
from sqlalchemy.orm import relationship
from datetime import datetime
import uuid

class User(Base):
    __tablename__ = "users"

    id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
    email = Column(String, unique=True, index=True, nullable=False)
    password_hash = Column(String, nullable=False)
    first_name = Column(String)
    last_name = Column(String)
    subscription_tier = Column(String, default="free") # free, pro, enterprise
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
    deleted_at = Column(DateTime, nullable=True)

    # Relationships
    documents = relationship("Document", back_populates="user")
    pipelines = relationship("Pipeline", back_populates="user")
```

## Vertex AI Orchestration Service

**apps/backend/app/services/orchestration\_service.py**

python



```
import httpx
import asyncio
import json
from app.config import settings

class VertexAIOrchestratorService:
    def __init__(self):
        self.base_url = f"https://{settings.VERTEX_LOCATION}-aiplatform.googleapis."
        self.project_id = settings.GCP_PROJECT_ID
        self.agent_id = settings.VERTEX_AGENT_ID

    async def call_orchestrator(self, input_data: dict, pipeline_stage: str) -> dict:
        """Call Vertex AI orchestrator with specific tool"""
        headers = {
            "Authorization": f"Bearer {await self._get_access_token()}",
            "Content-Type": "application/json"
        }

        payload = {
            "displayName": f"pipeline-{pipeline_stage}",
            "userInput": {
                "text": json.dumps(input_data)
            }
        }

        async with httpx.AsyncClient(timeout=60) as client:
            response = await client.post(
                f"{self.base_url}/projects/{self.project_id}/agents/{self.agent_id}",
                json=payload,
                headers=headers
            )

        return response.json()
```

```
async def _get_access_token(self) -> str:
    """Get GCP access token for authentication"""
    from google.auth.transport.requests import Request
    from google.oauth2.service_account import Credentials

    credentials = Credentials.from_service_account_file(
        settings.GCP_KEY_PATH,
        scopes=["https://www.googleapis.com/auth/cloud-platform"]
    )
    credentials.refresh(Request())
    return credentials.token
```

## Rate Limiting Middleware

**apps/backend/app/middleware/rate\_limit.py**

python

```
from fastapi import Request, HTTPException
from starlette.middleware.base import BaseHTTPMiddleware
import redis
import asyncio

class RateLimitMiddleware(BaseHTTPMiddleware):
    def __init__(self, app):
        super().__init__(app)
        self.redis_client = redis.Redis(
            host="localhost", port=6379, db=0, decode_responses=True
        )

    async def dispatch(self, request: Request, call_next):
        # Get user ID from JWT token (assuming already verified)
        user_id = request.state.user_id

        # Get current limit
        key = f"ratelimit:{user_id}"
        current = self.redis_client.incr(key)

        # Set expiration on first request
        if current == 1:
            self.redis_client.expire(key, 60) # 1 minute window

        # Check limit (100 requests per minute)
        if current > 100:
            raise HTTPException(
                status_code=429,
                detail="Rate limit exceeded",
                headers={"Retry-After": "60"}
            )

        response = await call_next(request)
```

```
response.headers["X-RateLimit-Remaining"] = str(100 - current)
return response
```

---

## Frontend (React/Next.js)

### API Client Setup

**apps/frontend-web/src/services/api.ts**

typescript

```
import axios, { AxiosInstance } from 'axios';

const api: AxiosInstance = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000/api',
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor: Add auth token
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor: Handle errors and token refresh
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      // Try to refresh token
      const refreshToken = localStorage.getItem('refresh_token');
      if (refreshToken) {
        const { data } = await axios.post(`${api.defaults.baseURL}/auth/refresh`, {
          refresh_token: refreshToken,
        });
        localStorage.setItem('access_token', data.access_token);
        // Retry original request
        return api(error.config);
      }
    }
  }
);
```

```
    }  
    return Promise.reject(error);  
  }  
);  
  
export default api;
```

## State Management (Zustand)

**apps/frontend-web/src/store/authStore.ts**

typescript

```
import { create } from 'zustand';
import api from '@services/api';

interface User {
  id: string;
  email: string;
  first_name: string;
}

interface AuthStore {
  user: User | null;
  loading: boolean;
  error: string | null;
  login: (email: string, password: string) => Promise<void>;
  signup: (email: string, password: string, name: string) => Promise<void>;
  logout: () => void;
}

export const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  loading: false,
  error: null,

  login: async (email, password) => {
    set({ loading: true, error: null });
    try {
      const { data } = await api.post('/auth/login', { email, password });
      localStorage.setItem('access_token', data.access_token);
      localStorage.setItem('refresh_token', data.refresh_token);
      set({ user: data.user, loading: false });
    } catch (error) {
      set({ error: error.message, loading: false });
    }
  }
})
```

```
    },  
  
    logout: () => {  
      localStorage.removeItem('access_token');  
      localStorage.removeItem('refresh_token');  
      set({ user: null });  
    },  
  }));
```

## WebSocket for Real-Time Updates

**apps/frontend-web/src/hooks/useInsightProgress.ts**

typescript



```
import { useEffect, useState } from 'react';

interface ProgressUpdate {
  stage: string;
  status: 'pending' | 'processing' | 'completed' | 'failed';
  message: string;
}

export function useInsightProgress(requestId: string) {
  const [progress, setProgress] = useState<ProgressUpdate[]>([]);
  const [isConnected, setIsConnected] = useState(false);

  useEffect(() => {
    if (!requestId) return;

    const token = localStorage.getItem('access_token');
    const ws = new WebSocket(
      `${process.env.NEXT_PUBLIC_WS_URL}/ws/${requestId}?token=${token}`
    );

    ws.onopen = () => setIsConnected(true);
    ws.onmessage = (event) => {
      const update = JSON.parse(event.data);
      setProgress(prev => [...prev, update]);
    };
    ws.onclose = () => setIsConnected(false);

    return () => ws.close();
  }, [requestId]);

  return { progress, isConnected };
}
```

---

## Terraform Infrastructure

### GCP Cloud Run Deployment

**infra/terraform/main.tf**

```
hcl
```

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
      version = "~> 5.0"
    }
  }
  backend "gcs" {
    bucket = "insight-to-action-tfstate"
  }
}

provider "google" {
  project = var.gcp_project_id
  region  = var.gcp_region
}

# Cloud Run service for backend
resource "google_cloud_run_service" "backend" {
  name      = "insight-to-action-backend"
  location = var.gcp_region

  template {
    spec {
      containers {
        image = google_artifact_registry_docker_repository.repo.repository_url

        env {
          name = "DATABASE_URL"
          value = google_sql_database_instance.postgres.private_ip_address
        }

        env {
```

```

        name = "REDIS_URL"
        value = google_memorystore_redis_instance.cache.host
    }

    resources {
        limits = {
            cpu      = "2"
            memory   = "2Gi"
        }
    }
}

    service_account_email = google_service_account.backend.email
    timeout_seconds       = 300
}

}

traffic {
    percent      = 100
    latest_revision = true
}

}

# Cloud SQL PostgreSQL instance
resource "google_sql_database_instance" "postgres" {
    name          = "insight-to-action-db"
    database_version = "POSTGRES_15"

    settings {
        tier      = "db-custom-4-16"
        disk_size = 100

        database_flags {

```

```

        name = "max_connections"
        value = "100"
    }

    location_preference {
        zone = "${var.gcp_region}-a"
    }

    backup_configuration {
        enabled = true
        start_time = "03:00"
        backup_retention_settings {
            retained_backups = 30
        }
    }
}

deletion_protection = true
}

# Cloud Memorystore Redis
resource "google_memorystore_redis_instance" "cache" {
    name = "insight-to-action-cache"
    memory_size_gb = 2
    tier = "standard"

    redis_version = "7.0"
    region = var.gcp_region
    location_id = "${var.gcp_region}-a"
    authorized_network = google_compute_network.vpc.id
}

# Artifact Registry for Docker images

```

```
resource "google_artifact_registry_docker_repository" "repo" {  
  location      = var.gcp_region  
  repository_id = "insight-to-action"  
  description   = "Docker repository for Insight-to-Action Engine"  
}
```

---

## Docker Configuration

### Dockerfile

**infra/docker/Dockerfile**

```
dockerfile
```

```
# Builder stage
FROM python:3.12-slim as builder

WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Runtime stage
FROM python:3.12-slim

WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12/site-packages /usr/local/lib/python3.12/site-packages
COPY apps/backend/ .

ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

## Docker Compose

### docker-compose.yml

yaml

```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: insight_to_action
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dev"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build: .
```



```
ports:
  - "8000:8000"
environment:
  DATABASE_URL: postgresql://dev:dev@postgres:5432/insight_to_action
  REDIS_URL: redis://redis:6379
  GCP_PROJECT_ID: ${GCP_PROJECT_ID}
  VERTEX_AGENT_ID: ${VERTEX_AGENT_ID}
depends_on:
  postgres:
    condition: service_healthy
  redis:
    condition: service_healthy
volumes:
  - ./apps/backend:/app
  command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

volumes:
  postgres_data:
  redis_data:
```

---

## GitHub Actions CI/CD

`/.github/workflows/deploy.yml`

```
yaml
```

```
name: Build and Deploy

on:
  push:
    branches: [main, staging]

env:
  GCP_PROJECT_ID: ${ secrets.GCP_PROJECT_ID }
  GCP_REGION: us-central1

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.12'

      - name: Install dependencies
        working-directory: apps/backend
        run: |
          pip install -r requirements.txt

      - name: Run tests
        working-directory: apps/backend
        run: pytest tests/ --cov=app --cov-report=xml

      - name: Upload coverage
        uses: codecov/codecov-action@v3
```

```
build:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3

    - name: Set up Cloud SDK
      uses: google-github-actions/setup-gcloud@v1
      with:
        service_account_key: ${ secrets.GCP_SA_KEY }
        project_id: ${ env.GCP_PROJECT_ID }

    - name: Build and push Docker image
      run: |
        gcloud builds submit --tag gcr.io/${ env.GCP_PROJECT_ID }/insight-to-ac

deploy:
  needs: build
  runs-on: ubuntu-latest
  steps:
    - name: Deploy to Cloud Run
      run: |
        gcloud run deploy insight-to-action-backend \
          --image gcr.io/${ env.GCP_PROJECT_ID }/insight-to-action:${ github.s
          --region ${ env.GCP_REGION } \
          --platform managed \
          --allow-unauthenticated
```

---

## Configuration Templates

### Environment Variables (.env.example)

bash

```
# Application
APP_NAME=insight-to-action
ENVIRONMENT=development
DEBUG=True

# Database
DATABASE_URL=postgresql://dev:dev@localhost:5432/insight_to_action
DATABASE_POOL_SIZE=10

# Redis
REDIS_URL=redis://localhost:6379

# JWT
SECRET_KEY=your-super-secret-key-change-in-production
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7

# GCP
GCP_PROJECT_ID=your-project-id
GCP_REGION=us-central1
GCP_KEY_PATH=/path/to/service-account-key.json
VERTEX_LOCATION=us-central1
VERTEX_AGENT_ID=your-agent-id

# CORS
CORS_ORIGINS=http://localhost:3000,http://localhost:3001

# API Keys (if needed)
OPENAI_API_KEY=
ANTHROPIC_API_KEY=

# Rate Limiting
```

```
RATE_LIMIT_PER_MINUTE=100
```

```
RATE_LIMIT_PER_HOUR=1000
```

---

## Database Migrations

**apps/backend/alembic/env.py** (partial)

```
python
```

```
from alembic import context
from sqlalchemy import engine_from_config, pool
from app.models import Base

def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode."""
    configuration = context.config
    sqlalchemy_url = configuration.get_main_option("sqlalchemy.url")

    context.configure(
        url=sqlalchemy_url,
        target_metadata=Base.metadata,
        literal_binds=True
    )

    with context.begin_transaction():
        context.run_migrations()

def run_migrations_online() -> None:
    """Run migrations in 'online' mode."""
    configuration = context.config

    connectable = engine_from_config(
        configuration.get_section(configuration.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with connectable.connect() as connection:
        context.configure(
            connection=connection,
            target_metadata=Base.metadata
        )
```

```
with context.begin_transaction():
    context.run_migrations()
```

---

## Testing Setup

### Unit Test Example

**apps/backend/tests/unit/test\_auth\_service.py**

```
python

import pytest
from app.services.auth_service import AuthService

class TestAuthService:
    def test_hash_password(self):
        password = "test123"
        hashed = AuthService.hash_password(password)
        assert hashed != password
        assert AuthService.verify_password(password, hashed)

    def test_create_access_token(self):
        token = AuthService.create_access_token("user-123")
        decoded = AuthService.verify_token(token)
        assert decoded["sub"] == "user-123"

    def test_invalid_token(self):
        decoded = AuthService.verify_token("invalid-token")
        assert decoded is None
```



## Integration Test Example

`apps/backend/tests/integration/test_auth_routes.py`

```
python
```

```
import pytest
from httpx import AsyncClient
from app.main import app

@pytest.mark.asyncio
async def test_signup():
    async with AsyncClient(app=app, base_url="http://test") as client:
        response = await client.post(
            "/api/auth/signup",
            json={
                "email": "test@example.com",
                "password": "password123",
                "first_name": "Test",
                "last_name": "User"
            }
        )
        assert response.status_code == 201
        assert "access_token" in response.json()

@pytest.mark.asyncio
async def test_login():
    async with AsyncClient(app=app, base_url="http://test") as client:
        # First signup
        await client.post("/api/auth/signup", json={...})

        # Then login
        response = await client.post(
            "/api/auth/login",
            json={"email": "test@example.com", "password": "password123"}
        )
        assert response.status_code == 200
        assert "access_token" in response.json()
```

---

## Monitoring & Logging

### Structured Logging Setup

`apps/backend/app/utils/logging.py`

```
python
```

```
import json
import logging
from pythonjsonlogger import jsonlogger

def setup_logging():
    logger = logging.getLogger()

    # JSON formatter for structured logging
    formatter = jsonlogger.JsonFormatter(
        '%(timestamp)s %(level)s %(name)s %(message)s'
    )

    handler = logging.StreamHandler()
    handler.setFormatter(formatter)
    logger.addHandler(handler)
    logger.setLevel(logging.INFO)

    return logger

# Usage in request middleware
logger = setup_logging()

logger.info("Request received", extra={
    "user_id": user_id,
    "method": request.method,
    "path": request.url.path,
    "correlation_id": correlation_id
})
```

---

## Production Checklist

- ☐ Set `DEBUG=False` in production
- ☐ Use strong `SECRET_KEY` (min 32 chars, random)
- ☐ Configure proper CORS origins
- ☐ Enable HTTPS only
- ☐ Set up database backups
- ☐ Configure Cloud Logging
- ☐ Set up monitoring alerts
- ☐ Enable audit logging
- ☐ Configure rate limiting
- ☐ Set up incident response procedures
- ☐ Load test before go-live
- ☐ Create runbooks for common issues
- ☐ Document API changes
- ☐ Plan disaster recovery

---

## End of Implementation Guide